

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores



CE 1108 | Compiladores e Intérpretes

Proyecto grupal 1

Compilador de Código Binario

Documentación

Estudiantes

Ana Melissa Vásquez Rojas

Darío Garro Moya

Jose Pablo Fernández Rojas

Isaac Somarribas Montero

Profesor:

Jason Leitón Jiménez

30 de Abril, 2026

Índice

1.1 Especificación de la Arquitectura	3
1.1.1. Tamaño de Palabra e Instrucción	3
1.1.1.1. Formatos de Instrucción	3
1.1.2. Conjunto de Registros	3
1.1.2.1. Registro de Estado (SR) y Bóveda de Llaves	4
1.1.3. Organización de Memoria	4
1.1.4. Tabla Completa de Instrucciones	4
1.1.4.1. ALU Registro–Registro (R-Type, opcode = 1100)	4
1.1.4.2. Aritmética con Inmediato (I-Type)	5
1.1.4.3. Acceso a Memoria	5
1.1.4.4. Saltos Condicionales (B-Type)	6
1.1.4.5. Saltos Incondicionales	6
1.1.4.6. Instrucciones Criptográficas TEA (V-Type, requieren AUTH=1)	6
1.1.4.7. Instrucciones de Bóveda / Autenticación (V-Type, opcode = 1101/1110)	7
1.2 Especificación del Lenguaje de Entrada	8
1.2.1. Definición de Tokens	8
1.2.1.1. Palabras Reservadas	8
1.2.1.2. Operadores	8
1.2.1.3. Literales	8
1.2.1.4. Expresiones regulares o regla léxica de cada token	9
1.2.2. Sintaxis de Elementos Básicos	10
1.2.2.1. Declaración de Variables y Constantes	10
1.2.2.2. Tipos de Dato	10
1.2.2.3. Asignaciones y Expresiones	10
1.2.3. Sintaxis de Estructuras de Control y Funciones	10
1.2.3.1. Condicional: <code>si</code> / <code>sino</code>	10
1.2.3.2. Ciclo <code>mientras</code>	10
1.2.3.3. Ciclo <code>para</code>	11
1.2.3.4. Declaración y Llamada de Funciones	11
1.2.4. Anotaciones y Bóveda	11
1.2.5. Importación de Módulos y Ámbito de Variables	12
1.3 Descripción de Algoritmos Clave	13
1.3.1. Análisis Léxico (Lexer)	13
1.3.2. Análisis Sintáctico y Construcción del AST	13
1.3.3. Construcción de Tablas de Símbolos	14
1.3.4. Generación de Ensamblador (Fase 4)	15
1.3.5. Cálculo de Saltos y Resolución de Referencias (Fase 5)	15
1.3.6. Generación de Código Binario (Fase 6)	16
1.4 Ejemplos de Ejecución	17
1.4.1. Ejercicio 1: Cálculo de Factorial	17

1.4.1.1. Código fuente (<code>factorial.yeison</code>)	17
1.4.1.2. Ensamblador generado y resuelto (primeras instrucciones) . . .	17
1.4.1.3. Llamadas recursivas	18
1.4.2. Ejercicio 2: Búsqueda en arreglo	18
1.4.2.1. Código fuente (<code>busqueda.yeison</code>)	18
1.4.2.2. Ensamblador generado y resuelto (instrucciones principales) . .	19
1.4.2.3. Traza de ejecución (<code>busqueda.yeison</code>)	19
1.4.3. Ejercicio 3: Carga de Llave Criptográfica (<code>vault.yeison</code>)	20
1.4.3.1. Código fuente (<code>vault.yeison</code>)	20
1.4.3.2. Ensamblador generado y resuelto (instrucciones principales) . .	21
1.4.3.3. Traza de ejecución (<code>vault.yeison</code>)	22
1.5 Manejo de Casos Especiales	22
1.5.1. Saltos Anidados (Condicionales dentro de Ciclos)	22
1.5.2. Llamadas a Función y Retorno de Valores	23
1.5.3. Conflictos de Variables en Diferentes Ámbitos (Shadowing)	23
1.5.4. Asignación de Memoria: Locales vs. Globales	24
1.5.5. Multiplicación sin Instrucción MUL	24
1.5.6. Carga de Constantes de 32 bits con LLI	24
Resumen: Pipeline del Compilador	25

1.1 Especificación de la Arquitectura

1.1.1 Tamaño de Palabra e Instrucción

La arquitectura TEA-ISA utiliza instrucciones de 23 bits, almacenadas de forma alineada en palabras de 32 bits (4 bytes) en little-endian. Cada instrucción ocupa exactamente 4 bytes en memoria, aunque solo 23 bits son significativos (bits [22:0]). El ancho de datos, el Contador de Programa (PC) y el Registro de Estado (SR) son de 32 bits.

Parámetro	Valor
Ancho de instrucción	23 bits
Ancho de datos	32 bits
Registros de propósito general	16 (R0–R15, 4-bit index)
Program Counter (PC)	32 bits
Status Register (SR)	32 bits (flags y estado de autenticación)
Almacenamiento por instrucción	32 bits (word-aligned, little-endian)
Campos de inmediato	32 bits (11 bits (I/S/B-type), 15 bits (J/JR-type))
Memoria mínima	64 KB
Endianness	Little-endian

1.1.1.1 Formatos de Instrucción

TEA-ISA define 7 formatos de instrucción. El campo opcode de 4 bits (bits [22:19]) determina el tipo:

Formato	Bits [22:19]	Campos	Uso
R-Type	opcode(4)	rd(4) rs1(4) rs2(4) funct(3) res(4)	ALU registro-registro
I-Type	opcode(4)	rd(4) rs1(4) imm(11)	ADDI, LLI, LOAD
S-Type	opcode(4)	rs2(4) rs1(4) offset(11)	STORE
B-Type	opcode(4)	rs1(4) rs2(4) offset(11)	BEQ, BNE, BGE, BGT
J-Type	opcode(4)	rd(4) offset(15)	JAL (salto con enlace)
JR-Type	opcode(4)	rs1(4) res(15)=0	JR, HALT, NOP
V-Type	opcode(4)	rd(4) rs1(4) ki(2) funct(9)	Bóveda y TEA

1.1.2 Conjunto de Registros

TEA-ISA dispone de 16 registros de propósito general (R0–R15), cada uno de 32 bits, con la siguiente convención de uso:

Registro	Alias	Propósito
R0	zero	Siempre contiene el valor 0; escrituras son ignoradas
R1	ra	Return Address: dirección de retorno de subrutinas
R2	sp	Stack Pointer: apunta al tope de la pila
R3	fp	Frame Pointer: apunta a la base del frame actual
R4-R7	a0-a3	Argumento de función / valores de retorno
R8-R11	t0-t3	Registros temporales
R12-R15	s0-s3	Registro guardado (caller-saved)

1.1.2.1 Registro de Estado (SR) y Bóveda de Llaves

Bit(s)	Nombre	Descripción
SR[0]	AUTH	Usuario autenticado (1 = sesión activa)
SR[1]	VF	Visible Flag: resultado de AUTHCHK
SR[2]	SEC_EXC	Security Exception Flag
SR[5:3]	SEC_CAUSE	Código de causa: 001=no-auth, 010=pwd-mal, 011=token-mal
SR[31:6]	—	Reservado

La Key Vault almacena 4 llaves de 128 bits (indexadas por 2 bits $k_i=0..3$), cada llave compuesta por 4 palabras de 32 bits ($k_0..k_3$). Las llaves nunca están expuestas en registros de propósito general.

1.1.3 Organización de Memoria

La arquitectura dispone de un espacio de dirección mínimo de 64 KB. Las instrucciones se escriben directamente desde el byte 0, y el PC arranca en 0x0000.

Segmento	Dirección base	Descripción
Código (Text)	0x0000	Instrucciones generadas. PC inicia en 0x0000.
Datos globales	0x0100	Variables y constantes globales declaradas con <code>var/const</code> .
Stack (Pila)	0xE000 (base)	Crece hacia abajo. SP se inicializa en 0xE000.

El compilador asigna automáticamente direcciones a las variables globales a partir de `DATA_BASE = 0x0100`, alineadas a múltiplos de 4 bytes. Las variables locales se asignan como offsets negativos respecto al SP, creciendo de 4 en 4.

1.1.4 Tabla Completa de Instrucciones

1.1.4.1 ALU Registro–Registro (R-Type, opcode = 1100)

Mnemónico	funct	Operación	Ejemplo
add rd, rs1, rs2	000	$rd = rs1 + rs2$	add r4, r5, r6
sub rd, rs1, rs2	001	$rd = rs1 - rs2$	sub r4, r5, r6
and rd, rs1, rs2	010	$rd = rs1 \& rs2$	and r4, r5, r6
or rd, rs1, rs2	011	$rd = rs1 rs2$	or r4, r5, r6
xor rd, rs1, rs2	100	$rd = rs1 \wedge rs2$	xor r4, r5, r6
sll rd, rs1, rs2	101	$rd = rs1 \ll rs2[4:0]$	sll r4, r5, r6
srl rd, rs1, rs2	110	$rd = rs1 \gg rs2[4:0]$ (lógico)	srl r4, r5, r6
sra rd, rs1, rs2	111	$rd = rs1 \gg rs2[4:0]$ (aritmético)	sra r4, r5, r6

1.1.4.2 Aritmética con Inmediato (I-Type)

Mnemónico	Opcode	Operación	Ejemplo
addi rd, rs1, imm	0001	$rd = rs1 + \text{sign_ext}(\text{imm}[10:0])$	addi r4, r4, -4
lli rd, imm8, pos	0010	Carga byte en posición 0-3 de rd	lli r5, 0xB9, 0

LLI usa `imm[10:9]` como selector de byte (0=bits[7:0], 1=bits[15:8], 2=bits[23:16], 3=bits[31:24]) e `imm[7:0]` como valor. Se requieren hasta 4 instrucciones LLI para cargar una constante de 32 bits.

1.1.4.3 Acceso a Memoria

Mnemónico	Opcode	Operación	Ejemplo
load rd, offset(rs1)	0011	$rd = \text{mem}[rs1 + \text{sign_ext}(\text{offset})]$	load r8, 0(r2)
store rs2, offset(rs1)	0100	$\text{mem}[rs1 + \text{sign_ext}(\text{offset})] = rs2$	store r9, 0(r2)

1.1.4.4 Saltos Condicionales (B-Type)

Mnemónico	Opcode	off[10]	Condición	Ejemplo
beq rs1, rs2, off	0101	0	si $rs1 == rs2$: $PC += off \times 4$	beq r8, r0, 4
bne rs1, rs2, off	0101	1	si $rs1 \neq rs2$: $PC += off \times 4$	bne r8, r9, -3
bge rs1, rs2, off	0110	0	si $rs1 \geq rs2$: $PC += off \times 4$ (signed)	bge r8, r9, 5
bgt rs1, rs2, off	0110	1	si $rs1 > rs2$: $PC += off \times 4$ (signed)	bgt r8, r9, 5

1.1.4.5 Saltos Incondicionales

Mnemónico	Opcode	Operación	Ejemplo
jal rd, offset	0111	$rd = PC+4$; $PC = PC + sign_ext(offset \times 4)$	jal r1, main
jr rs1	1000	$PC = rs1$	jr r1
nop	0000	No operación	nop
halt	1001	Detiene la ejecución	halt

1.1.4.6 Instrucciones Criptográficas TEA (V-Type, requieren AUTH=1)

Mnemónico	Opcode	Operación	Ejemplo
tea_add1 rd, rs1, ki	1010	$rd = (rs1 \ll 4) + K[ki]$	tea_add1 r4, r5, 0
tea_add2 rd, rs1, ki	1011	$rd = (rs1 \gg 5) + K[ki]$	tea_add2 r4, r5, 1

1.1.4.7 Instrucciones de Bóveda / Autenticación (V-Type, opcode = 1101/1110)

Mnemónico	funct[8:0]	Req. AUTH	Descripción
setpwd(uid, pwd)	000000000	No	Establece contraseña para usuario uid
login(uid, pwd)	000000001	No	Autentica uid/pwd → AUTH=1, SIC=0
logout()	000000010	No	Limpia AUTH, VF, SEC_EXC, SEC_CAUSE
authorize(uid, token)	000000011	No	Autenticación por token → AUTH=1
vkload(rs1, ki)	000000100	Sí	Carga palabra rs1 en K[ki] de la llave activa
vkinv(ki)	000000101	Sí	Zeroiza (borra) la llave K[ki] del vault
authchk()	000000011	No	Copia AUTH en VF (opcode=1110)

El Session Instruction Counter (SIC) se reinicia a 0 en cada login/authorize. Cuando llega a 100 instrucciones con AUTH=1, se ejecuta logout() automáticamente.

1.2 Especificación del Lenguaje de Entrada

El lenguaje de entrada se denomina **Yeison** (extensión `.yeison`). Es un lenguaje imperativo de tipado estático con soporte para funciones, estructuras de control, arreglos y acceso directo a las instrucciones criptográficas de la arquitectura TEA-ISA.

1.2.1 Definición de Tokens

1.2.1.1 Palabras Reservadas

Categoría	Palabras reservadas
Control de flujo	si, sino, mientras, para, retorna, func
Declaraciones	var, const, importar
Tipos de dato	int, uint, bool, string, real, void
Literales booleanos	true, false
Operadores lógicos	and, or, not
Bóveda / Auth	login, logout, setpwd, authchk, authorize, vkload, vkinv
Anotaciones	@code, @boveda

1.2.1.2 Operadores

Tipo	Símbolos
Aritméticos	+ - * / % **
Relacionales	== != < > <= >=
Lógicos	and or not
Bits	<< >> & ^ ~
Asignación	=
Tipo de retorno	::

Los **delimitadores** son: () { } [] : , y el terminador de sentencia ' (apóstrofo). El lenguaje soporta comentarios de línea (#) y de bloque (## ... ##).

1.2.1.3 Literales

Tipo	Ejemplo	Token generado
Entero decimal	42	INTEGER
Entero sin signo	42u	INTEGER_U
Hexadecimal	0xFF	HEXADECIMAL
Hexadecimal uint	0xFFu	HEXADECIMAL_U
Real	3.14	REAL_NUMBER
Cadena	"hola"	STRING_LITERAL
Booleano	true / false	TRUE / FALSE

1.2.1.4 Expresiones regulares o regla léxica de cada token

Token	Ejemplo	Definición / Regla
Identificador	<code>miVariable</code>	Inicia con letra minúscula. Puede contener letras, dígitos y guion bajo (<code>_</code>). Soporta <code>camelCase</code> y <code>snake_case</code> .
Entero	<code>42, -15</code>	Secuencia de uno o más dígitos, opcionalmente precedida por signo negativo.
Real	<code>3.14, -0.5</code>	Secuencia de dígitos con punto decimal, opcionalmente precedida por signo negativo.
Cadena	<code>"hola mundo"</code>	Texto delimitado por comillas dobles. No se permiten comillas dobles dentro del contenido.
Hexadecimal	<code>0xFF, 0xA1</code>	Inicia con <code>0x</code> seguido de uno o más dígitos hexadecimales (<code>0-9, a-f, A-F</code>).
Booleano	<code>true, false</code>	Se reconoce literalmente como <code>true</code> o <code>false</code> .
Palabras reservadas	<code>si, mientras, func</code>	Se reconocen literalmente: <code>si, sino, mientras, para, retorna, func, var, true, false, const, and, or, not, int, bool, string, real, uint, void</code> .
Operadores aritméticos	<code>+ - * ** / %</code>	Se reconocen literalmente los símbolos.
Operadores relacionales	<code>== != <> <= >=</code>	Se reconocen literalmente los símbolos.
Operador de asignación	<code>=</code>	Se reconoce literalmente el símbolo.
Operadores lógicos	<code>and, or, not</code>	Se reconocen literalmente las palabras.
Operadores a nivel de bit	<code><< ^ ~ & </code>	Se reconocen literalmente los símbolos.
Delimitadores	<code>{ } () [] ,</code>	Se reconocen literalmente los símbolos.
Operador de tipo	<code>:</code>	Se reconoce el símbolo para asociar identificadores con tipos.
Operador de retorno	<code>::</code>	Se reconoce el símbolo para indicar el tipo de retorno de una función.
Terminador de instrucción	<code>'</code>	Se reconoce literalmente el símbolo.
Comentario de línea	<code># comentario</code>	Inicia con <code>#</code> y termina al final de la línea.
Comentario de bloque	<code>## comentario ##</code>	Inicia con <code>##</code> y termina con <code>##</code> .

1.2.2 Sintaxis de Elementos Básicos

1.2.2.1 Declaración de Variables y Constantes

```
var nombre : tipo = expresion'
const nombre : tipo = expresion'

# Ejemplos:
var x : int = 0'
var datos : [2]uint = [0x00000000, 0x00000000]'
const DELTA : uint = 0x9E3779B9'
```

1.2.2.2 Tipos de Dato

Tipo	Descripción
int	Entero con signo de 32 bits
uint	Entero sin signo de 32 bits
bool	Booleano (true / false)
real	Punto flotante de 32 bits
string	Cadena de texto (solo para literales)
void	Tipo de retorno vacío para funciones
[N]tipo	Arreglo de N elementos del tipo indicado

1.2.2.3 Asignaciones y Expresiones

```
nombre = expresion'
arr[i] = expresion'

# Precedencia de operadores (menor a mayor):
# or -> and -> not -> comparacion -> bits -> +/- -> *//% -> ** -> -/~
  -> atomo
```

1.2.3 Sintaxis de Estructuras de Control y Funciones

1.2.3.1 Condicional: si / sino

```
si (condicion) {
    # bloque
} sino {
    # bloque (opcional)
}
```

1.2.3.2 Ciclo mientras

```
mientras (condicion) {
```

```
# cuerpo
}
```

1.2.3.3 Ciclo para

```
para (init' condicion' update') {
    # cuerpo
}

# Ejemplo:
para (i = 0' i < 32' i = i + 1') {
    suma = suma + DELTA'
}
```

1.2.3.4 Declaración y Llamada de Funciones

```
func nombre(param1 : tipo1, param2 : tipo2) :: tipo_retorno {
    # cuerpo
    retorna valor'    # o retorna' si es void
}

# Ejemplo:
func factorial(n : int) :: int {
    si (n <= 1) {
        retorna 1'
    } sino {
        retorna n * factorial(n - 1)'
    }
}

# Llamada:
func main()::void{

    var n : int = 5'

    factorial(n)'

}
```

1.2.4 Anotaciones y Bóveda

Las anotaciones @code y @boveda dividen el código en secciones. Las instrucciones de autenticación (login, logout, vkload, etc.) y las criptográficas tea_add1/tea_add2 solo pueden usarse dentro de bloques @boveda.

```
@boveda
login(0, 1234)'
vkload(k0, 0)'
```

```
@code
# código normal aquí
```

1.2.5 Importación de Módulos y Ámbito de Variables

El lenguaje soporta importación de módulos con la palabra clave `importar`:

```
importar cifrado'
cifrado.tea_cifrar(addr)' # llama a la función del módulo
```

El ámbito sigue scoping léxico: las variables locales de una función ocultan a las globales del mismo nombre (shadowing). Cada función abre su propio ámbito; las variables locales no son accesibles fuera de ella.

1.3 Descripción de Algoritmos Clave

1.3.1 Análisis Léxico (Lexer)

El lexer está implementado en `lexer.py` como un Autómata de Estado Finito Determinista (DFA) construido manualmente. El método `next_token()` invoca primero `skip()` para descartar espacios y comentarios, y luego asigna un sub-autómata según el carácter actual. Si la cadena actual coincide con una palabra reservada se asigna a esta, de lo contrario se asigna como identificador. Subautomatas:

- **Letra o guión bajo** → `read_identifier()`: consume alfanuméricos y guiones bajos; busca en la tabla `KEYWORDS`. Soporta identificadores con punto (`módulo.símbolo`) para imports.
- **Dígito** → `read_number()`: maneja decimales, hexadecimales (`0x...`) y reales. Los sufijos `u` producen tokens `_U`.
- **Comilla** → `read_string()`: acumula hasta la comilla de cierre; maneja secuencias de escape (`\n`, `\t`, etc.).
- **@** → `read_annotation()`: valida contra `VALID_ANNOTATIONS = {'boveda', 'code'}`.
- **Otro** → `read_operator()`: intenta operadores de dos caracteres primero (`**`, `==`, `!=`, `<=`, `>=`, `<<`, `>>`, `::`).

El manejo de errores usa `panic_mode()`: avanza hasta encontrar un carácter de sincronización (`SYNC`), registra el error y continúa el análisis.

1.3.2 Análisis Sintáctico y Construcción del AST

El parser (`parser.py`) es un analizador descendente recursivo con un token de lookahead `LL(1)` en la mayoría de casos. Detecta anotaciones `@code/@boveda` que separan el programa en `SectionBlocks`.

La jerarquía de precedencia de expresiones (de menor a mayor):

Nivel	Método	Operadores
1 (menor)	<code>parse_or()</code>	<code>or</code>
2	<code>parse_and()</code>	<code>and</code>
3	<code>parse_not()</code>	<code>not</code> (unario)
4	<code>parse_comparison()</code>	<code>== != <><= >=</code>
5	<code>parse_bits()</code>	<code><< >> & ^</code>
6	<code>parse_suma()</code>	<code>+</code> <code>-</code>
7	<code>parse_termino()</code>	<code>*</code> <code>/</code> <code>%</code>
8	<code>parse_potencia()</code>	<code>**</code> (asociativo derecha)
9	<code>parse_negacion()</code>	<code>-</code> <code>~</code> (unario)
10 (mayor)	<code>parse_atom()</code>	literales, identificadores, llamadas, índices, <code>()</code>

Los nodos del AST están definidos en `ast_nodes.py`. El árbol captura la estructura jerárquica completa: `Program` luego `SectionBlock` luego `FunctionDeclaration` luego `IfStatement` luego `BinaryOp`, etc.

1.3.3 Construcción de Tablas de Símbolos

El análisis semántico (`semantic.py`) implementa un Visitor sobre el AST. Mantiene una `SymbolTable` basada en una pila de diccionarios (uno por ámbito):

- `open_scope(nombre)`: empuja un nuevo diccionario al entrar en función o bloque.
- `close_scope()`: saca el diccionario al salir del ámbito.
- `define(símbolo)`: registra en el ámbito actual; retorna `False` si ya existe (RS-02).
- `lookup(nombre)`: busca de adentro hacia afuera (scoping léxico, RS-01).

Las variables globales incrementan `data_ptr` desde `DATA_BASE = 0x0100`; las locales y parámetros se asignan como offset relativo al SP, alineados a 4 bytes con `align()`.

Reglas semánticas verificadas:

Regla	Descripción
RS-01	Todo identificador debe ser declarado antes de usarse
RS-02	No se puede declarar el mismo nombre dos veces en el mismo ámbito; <code>mem</code> es reservado
RS-03	Las constantes (<code>const</code>) no pueden ser asignadas después de su declaración
RS-04	Las funciones deben llamarse con el número correcto de argumentos
RS-05	Los tipos en asignaciones deben ser compatibles; <code>int</code> y <code>uint</code> son mutuamente compatibles
RS-06	Operadores de bits requieren enteros; operadores aritméticos requieren numéricos
RS-07	Operadores <code>and/or/not</code> requieren valores booleanos
RS-08	El tipo retornado debe coincidir con el declarado en la función
RS-09	Los índices de arreglos deben ser enteros
RS-10	Las funciones de bóveda solo pueden llamarse dentro de sección <code>@boveda</code>

1.3.4 Generación de Ensamblador (Fase 4)

El generador (`asmgen.py`) recorre el AST anotado usando el patrón Visitor en dos pasadas:

Pasada 1 — `_assign_function_addresses()`: estima el número de instrucciones de cada función para calcular su dirección de entrada correcta.

Pasada 2 — `generate()`: emite instrucciones en este orden:

- Prólogo: inicialización del SP (`lli r2, 0, 0 + lli r2, 0xE0, 1 → r2 = 0xE000`).
- Inicialización de variables globales (`load_immediate + store`).
- Salto a `main` con `jal r1, main + halt`.
- Cuerpo de cada función declarada.

La convención de llamadas usa R4–R7 para argumentos (a0–a3) y R4 para el valor de retorno. R1 (ra) guarda la dirección de retorno. La gestión de registros temporales mantiene un pool de R8–R11: `_alloc_reg()` obtiene uno libre y `_free_reg()` lo devuelve.

1.3.5 Cálculo de Saltos y Resolución de Referencias (Fase 5)

El resolver (`resolver.py`) opera sobre el ensamblador de la Fase 4 en dos pasadas:

Pasada 1 — `labels_direction()`: recorre todas las líneas y clasifica cada una. Las líneas que terminan con `':'` son etiquetas y registran su dirección actual. Cualquier otra línea es una instrucción y avanza el PC en 4 bytes.

Pasada 2 — `labels_rewrite()`: para cada instrucción de salto (`jal`, `jr`, `beq`, `bne`, `bge`, `bgt`), reemplaza nombres de etiquetas con el offset relativo numérico:

```
offset = (addr_label - addr_instruccion) / 4
```



```
# Ejemplo:
# Instruccion en 0x00: jal r1, main    -> main esta en 0x88
# offset = (0x88 - 0x00) / 4 = 34
# Resultado: jal r1, 34
```

Los offsets se expresan en palabras (no bytes), ya que la ISA multiplica el campo de offset $\times 4$ internamente. El matching usa expresiones regulares con *word boundaries* (`\b`) para evitar sustituciones parciales.

1.3.6 Generación de Código Binario (Fase 6)

El BinaryGen (`binary_gen.py`) toma el ensamblador resuelto y codifica cada instrucción según el formato TEA-ISA de 23 bits. Cada instrucción se almacena en 4 bytes en little-endian. El algoritmo por línea:

- Parsear la mnemónica y sus operandos.
- Determinar el tipo (R/I/S/B/J/JR/V) por la mnemónica.
- Extraer el opcode de la tabla `OPCODES`.
- Codificar cada campo en su posición de bit usando desplazamientos y OR.
- Convertir el entero de 23 bits a 4 bytes little-endian con `struct.pack('<I', valor)`.
- Escribir al archivo `.bin`; opcionalmente escribir línea hex en el `.mem`.

Formato del archivo `.mem`: El archivo de salida `.mem` contiene una instrucción por línea en el formato `HHHHHH // mnemónico operandos`, donde `HHHHHH` es la representación hexadecimal de los 3 bytes bajos de la instrucción codificada. Este archivo facilita la depuración sin necesidad de un desensamblador. Ejemplo real del `factorial.mem`:

```
110000 // lli r2, 0, 0
1102E0 // lli r2, 0xE0, 1
388022 // jal r1, 34
480000 // halt
```

Formato del archivo `.bin`: El archivo binario no incluye encabezado. Las instrucciones se escriben consecutivamente en little-endian desde el byte 0. El PC del procesador debe inicializarse en `0x0000` antes de ejecutar.

1.4 Ejemplos de Ejecución

1.4.1 Ejercicio 1: Cálculo de Factorial

El programa calcula el factorial de 5 (resultado esperado: 120) usando una función recursiva, con variables locales, condicionales y operaciones aritméticas

1.4.1.1 Código fuente (factorial.yeison)

```
# Test: Factorial de 5 - Salida esperada: 120

func factorial(n : int) :: int {
  si (n <= 1) {
    retorna 1'
  }
  sino {
    retorna n * factorial(n - 1)'
  }
}

func main() :: void {
  var n : int = 5'
  factorial(n)'
}
```

1.4.1.2 Ensamblador generado y resuelto (primeras instrucciones)

Hex	Instrucción	Descripción
110000	lli r2, 0, 0	Inicializar SP: byte 0 = 0x00
1102E0	lli r2, 0xE0, 1	SP[15:8] = 0xE0 → r2 = 0xE000
388022	jal r1, 34	Saltar a main (PC + 34×4)
480000	halt	Detener ejecución
642000	add r8, r4, r0	factorial: copiar n en t0
0C8001	addi r9, r0, 1	t1 = 1
34C405	bgt r8, r9, 5	si n > 1: saltar al caso recursivo
0C8001	addi r9, r0, 1	t1 = 1 (retorno base)
624800	add r4, r9, r0	a0 = 1
408000	jr r1	retornar
380019	jal r0, 25	saltar a multiplicación
...	...	(código de multiplicación y llamada recursiva)
408000	jr r1	retornar de main

1.4.1.3 Llamadas recursivas

Llamada	n	Retorna	SP
main $\rightarrow$ factorial(5)	5	—	0xDFF4
factorial(5) $\rightarrow$ factorial(4)	4	—	0xDFEC
factorial(4) $\rightarrow$ factorial(3)	3	—	0xDFE4
factorial(3) $\rightarrow$ factorial(2)	2	—	0xDFDC
factorial(2) $\rightarrow$ factorial(1)	1	1	0xDFD4
factorial(2) retorna	2	$2 \times 1 = 2$	0xDFDC
factorial(3) retorna	3	$3 \times 2 = 6$	0xDFE4
factorial(4) retorna	4	$4 \times 6 = 24$	0xDFEC
factorial(5) retorna	5	$5 \times 24 = 120$	0xDFF4

1.4.2 Ejercicio 2: Búsqueda en arreglo

Este programa busca la posición de un número en un arreglo.

1.4.2.1 Código fuente (busqueda.yeison)

```
@code
func buscar(datos : [5]int, objetivo : int) :: int {
    var i      : int = 0'
    var encontrado : int = 0'
    var posicion  : int = 0'
    mientras (i < 5) {
        si (datos[i] == objetivo) {
            encontrado = 1'
            posicion    = i'
            i = 5'      # forzar salida del ciclo
        } sino {
            i = i + 1'
        }
    }
    si (encontrado == 1) {
        retorna posicion'
    } sino {
        retorna 0'      # no encontrado
    }
}

func main() :: int {
    var datos : [5]int = [10, 25, 37, 42, 8]'
    var objetivo : int = 37'
    var pos : int = buscar(datos, objetivo)'
    retorna pos'
}
```

1.4.2.2 Ensamblador generado y resuelto (instrucciones principales)

Hex	Instrucción	Descripción
110000	lli r2, 0, 0	Inicializar SP: byte bajo = 0x00
1102E0	lli r2, 0xE0, 1	SP[15:8] = 0xE0 → r2 = 0xE000
388030	jal r1, 48	Saltar a main
480000	halt	Detener ejecución
— función <i>buscar(datos, objetivo)</i> —		
0917FC	addi r2, r2, -4 × 3	Reservar e inicializar i, encontrado, posicion = 0
1C1008	load r8, 8(r2)	while_start: cargar i
34C015	bge r8, r9, 21	Si $i \geq 5$: saltar a while_end
65C550	sll r11, r8, r10	Calcular $\&datos[i] = base + i*4$
1CC000	load r9, 0(r8)	Cargar datos[i]
2C4C08	bne r9, r8, 8	Si $datos[i] \neq objetivo$: saltar a else
0C0001	addi r8, r0, 1	encontrado = 1, posicion = i
0C0005	addi r8, r0, 5	Forzar i = 5 para salir del ciclo
380005	jal r0, 5	Saltar a endif
65C480	add r11, r8, r9	else: $i = i + 1$ y guardar
387FEA	jal r0, -22	Volver a while_start
2CDC06	bne r11, r9, 6	while_end: si no encontrado, saltar
624800	add r4, r9, r0	a0 = posicion (retorno)
09100C	addi r2, r2, 12 / jr r1	Liberar frame y retornar
0C8000	addi r9, r0, 0	else: a0 = 0 (no encontrado)
408000	jr r1	Liberar frame y retornar
— función <i>main()</i> —		
0917EC	addi r2, r2, -20	Reservar datos[5] en stack
0C800A	addi / store ×5	Inicializar datos = [10, 25, 37, 42, 8]
0C8025	addi r9, r0, 37	objetivo = 37; guardar en stack
209000	store r1, 0(r2)	Guardar dirección de retorno
38FFBD	jal r1, -67	Llamar buscar(datos, objetivo)
189000	load r1, 0(r2)	Restaurar retorno; a0 = resultado
09101C	addi r2, r2, 28 / jr r1	Liberar frame y retornar

1.4.2.3 Traza de ejecución (busqueda.yeison)

El arreglo es `datos = [10, 25, 37, 42, 8]` y `objetivo = 37`. La función `buscar` recorre el arreglo hasta encontrar el valor en el índice 2.

It.	i	datos[i]	encontrado	posicion	Acción
0	0	10	0	0	10 != 37 → i = 1
1	1	25	0	0	25 != 37 → i = 2
2	2	37	1	2	37 == 37 → i = 5 (salida)
3	5	—	1	2	i ≥ 5: salir del ciclo
encontrado == 1 → retorna posicion = 2					

El valor de retorno en `r4` = 2 corresponde al índice base 0 donde se encontró el 37. El SP se decrementa 12 bytes al entrar a `buscar` (3 variables locales × 4 bytes) y se restaura al retornar.

1.4.3 Ejercicio 3: Carga de Llave Criptográfica (`vault.yeison`)

El tercer programa demuestra el uso de funciones con llamadas a múltiples operaciones de bóveda, variables locales de 32 bits cargadas mediante `lli`, estructuras de control de sección (`@boveda`), y una función que llama a otra función (`main` llama a `vault`).

1.4.3.1 Código fuente (`vault.yeison`)

```
func vault():void{

    var token : uint = 0xDDAABBCF'

    @boveda
    authorize(0, token)'
    setpwd(0, 1234)'    # usuario 0, contraseña 1234
    login(0, 1234)'    # activar AUTH

    # Cargar llave de 128 bits en el vault (4 palabras de 32 bits)
    var k0 : uint = 0x01234567'
    var k1 : uint = 0x89ABCDEF'
    var k2 : uint = 0xFEDCBA98'
    var k3 : uint = 0x76543210'

    vkload(k0, 0)'    # K[0] = 0x01234567
    vkload(k1, 1)'    # K[1] = 0x89ABCDEF
    vkload(k2, 2)'    # K[2] = 0xFEDCBA98
    vkload(k3, 3)'    # K[3] = 0x76543210
}

func main():void {
    vault()'
}
```

1.4.3.2 Ensamblador generado y resuelto (instrucciones principales)

Hex	Instrucción	Descripción
110000	lli r2, 0, 0	Inicializar SP (byte bajo)
1102E0	lli r2, 0xE0, 1	Inicializar SP $\rightarrow$ r2 = 0xE000
388035	jal r1, 53	Llamar a <code>vault()</code>
480000	halt	Fin del programa
— función <code>vault()</code> —		
0917FC	addi r2, r2, -4	Reservar espacio en stack (token)
1400CF ... 1406DD	lli r8, ... $\times 4$	Construir 0xDDAABBCF en r8 (por bytes)
241000	store r8, 0(r2)	Guardar token en stack
1C9000	load r9, 0(r2)	Cargar token
6C4803	authorize r8, r9	Autorizar usuario con token
6CC000	setpwd r9, r8	Configurar contraseña (1234)
6C4801	login r8, r9	Activar autenticación
0917FC	addi r2, r2, -4 $\times 4$	Reservar espacio para llaves k0–k3
148067 ...	lli r9, ... $\times 4$	Construir k0 = 0x01234567
1480EF ...	lli r9, ... $\times 4$	Construir k1 = 0x89ABCDEF
148098 ...	lli r9, ... $\times 4$	Construir k2 = 0xFEDCBA98
148010 ...	lli r9, ... $\times 4$	Construir k3 = 0x76543210
249000	store r9, 0(r2) $\times 4$	Guardar llaves en stack
1C900C	load r9, 12(r2)	Cargar k0
684804	vkload r9, 0	Guardar en vault K[0]
1C9008	load r9, 8(r2)	Cargar k1
684A04	vkload r9, 1	Guardar en vault K[1]
1C9004	load r9, 4(r2)	Cargar k2
684C04	vkload r9, 2	Guardar en vault K[2]
1C9000	load r9, 0(r2)	Cargar k3
684E04	vkload r9, 3	Guardar en vault K[3]
091014	addi r2, r2, 20	Liberar stack frame
408000	jr r1	Retornar de <code>vault()</code>
— función <code>main()</code> —		
0917FC	addi r2, r2, -4	Guardar dirección de retorno
209000	store r1, 0(r2)	Push de r1
38FFCB	jal r1, -53	Llamar a <code>vault()</code>
189000	load r1, 0(r2)	Restaurar r1
091004	addi r2, r2, 4	Liberar stack
408000	jr r1	Retornar de <code>main()</code>

1.4.3.3 Traza de ejecución (vault.yeison)

El programa ejecuta las siguientes fases en secuencia. El SP inicia en 0xE000 y decrece conforme se reservan variables locales.

Fase	Instrucción clave	SP	Estado / Efecto
1	jal r1, 53	0xE000	Salto a main; r1 = PC_ret
2	store r1, 0(r2)	0xDFFC	Guardar r1 de main en pila
3	jal r1, -53	0xDFFC	Llamar vault(); r1 = ret_main
4	store r8, 0(r2)	0xDFF8	token = 0xDDAABBCF en pila
5	authorize r8,r9	0xDFF8	AUTH activado via token
6	setpwd r9, r8	0xDFF8	Contraseña uid=0 establecida
7	login r8, r9	0xDFF8	AUTH=1, SIC=0
8	store k0..k3	0xDFE8	4 llaves en pila (16 bytes)
9	vkload r9, 0..3	0xDFE8	K[0..3] cargadas en vault
10	addi r2, r2, 20	0xDFFC	Frame de vault liberado
11	jr r1	0xDFFC	Retornar a main
12	addi r2, r2, 4	0xE000	Frame de main liberado

Al finalizar, la Key Vault contiene la llave de 128 bits distribuida en cuatro registros internos: K[0]=0x01234567, K[1]=0x89ABCDEF, K[2]=0xFEDCBA98, K[3]=0x76543210. El SR tiene AUTH=1 activo, habilitando las instrucciones criptográficas TEA para operaciones posteriores.

1.5 Manejo de Casos Especiales

1.5.1 Saltos Anidados (Condicionales dentro de Ciclos)

El compilador genera etiquetas únicas con un contador global (`label_count`) durante el análisis semántico, garantizando que las etiquetas de diferentes bloques de control no colisionen:

```
# Para si anidado dentro de mientras:
# Semantico genera: while_start_0, while_end_1, else_2, endif_3

while_start_0:                # inicio del while
    <evaluar condicion>
    beq/bne ... while_end_1 # salir si la condicion es falsa
```

```

    <cuerpo del while>
    si_interno:
    <evaluar condicion interna>
    bge/bgt ... else_2      # saltar al else si falla
    <then block>
    jal r0, endif_3        # saltar al final del if
else_2:
    <else block>
endif_3:
    jal r0, while_start_0  # volver al inicio del while
while_end_1:

```

El Resolver (Fase 5) calcula los offsets relativos de forma independiente para cada instrucción de salto, sin importar el nivel de anidamiento.

1.5.2 Llamadas a Función y Retorno de Valores

El compilador implementa una convención de llamadas estándar para la arquitectura TEA-ISA:

Fase	Acciones del compilador
Pre-llamada (caller)	Guardar R1 en pila (<code>addi r2, r2, -4; store r1, 0(r2)</code>); guardar argumentos en R4–R7.
Llamada	<code>jal r1, nombre_función</code> ($R1 \leftarrow PC+4$; $PC \leftarrow \text{función}$).
Prólogo (callee)	Guardar variables locales en pila; decrementar SP.
Retorno de valor	Mover resultado a R4 (<code>add r4, resultado, r0</code>).
Epílogo (callee)	Restaurar SP (<code>addi r2, r2, +offset_local</code>); <code>jr r1</code> .
Post-llamada (caller)	Restaurar R1 de la pila (<code>load r1, 0(r2); addi r2, r2, 4</code>).

Las funciones con múltiples argumentos los reciben siempre en a0–a3 (R4–R7). Si la función llama a otras funciones, guarda R1 en la pila antes del `jal` para preservar su propia dirección de retorno.

1.5.3 Conflictos de Variables en Diferentes Ámbitos (Shadowing)

El semántico usa una pila de diccionarios (lexical scoping). Cuando se declara una variable local con el mismo nombre que una global, `lookup()` encuentra primero la local:

```

var x : int = 10'      # x global -> 0x0100

func foo() :: void {
    var x : int = 99'  # x local -> offset 0x00 en el frame de foo
    # Dentro de foo, 'x' se refiere a la local (offset 0x00)

```



```
# La global x en 0x0100 no es visible aqui
}
# Fuera de foo, 'x' vuelve a referirse a la global 0x0100
```

1.5.4 Asignación de Memoria: Locales vs. Globales

Tipo	Región	Dir. base	Crece	Acceso en ASM
Variable global	Segmento da- tos	0x0100	Ascendente	addi rX, r0, addr; store/load
Variable local/- param	Frame en pila	SP (0xE000)	Descendente	addi rX, r2, off; store/load
Constante nu- mérica	Inmediato	N/A	N/A	LLI (hasta 4 instruc- ciones)
Acceso mem[]	Directo	Cualquiera	N/A	load/store con direc- ción calculada

Todas las variables se alinean a múltiplos de 4 bytes. Los arreglos de N elementos ocupan $N \times 4$ bytes contiguos, accedidos mediante offsets calculados en tiempo de compilación o dinámicamente con `add + load/store`.

1.5.5 Multiplicación sin Instrucción MUL

TEA-ISA no incluye instrucción MUL nativa. El compilador implementa la multiplicación mediante sumas sucesivas, generando un bucle de ensamblador:

```
# a * b (donde b es el contador)
mul_loop_N:
    beq r_b, r0, mul_end_N    # si b == 0: terminar
    add r_acum, r_acum, r_a    # acum += a
    sub r_b, r_b, 1           # b -= 1
    jal r0, mul_loop_N        # repetir
mul_end_N:
```

1.5.6 Carga de Constantes de 32 bits con LLI

Dado que los campos inmediatos son de 11 bits, las constantes de 32 bits se cargan con hasta 4 instrucciones LLI, una por cada byte:

```
# Cargar DELTA = 0x9E3779B9:
lli r8, 0xB9, 0    # r8[7:0]    = 0xB9  -> r8 = 0x000000B9
lli r8, 0x79, 1    # r8[15:8]   = 0x79  -> r8 = 0x000079B9
lli r8, 0x37, 2    # r8[23:16]  = 0x37  -> r8 = 0x003779B9
lli r8, 0x9E, 3    # r8[31:24]  = 0x9E  -> r8 = 0x9E3779B9
```

Resumen: Pipeline del Compilador

Fase	Módulo	Entrada	Salida
1. Análisis Léxico	<code>lexer.py</code>	<code>.yeison</code> (texto)	Lista de Tokens
2. Análisis Sintáctico	<code>parser.py</code>	Lista de Tokens	AST (árbol sintáctico)
3. Análisis Semántico	<code>semantic.py</code>	AST	AST anotado + Tabla de Símbolos
4. Generación de ASM	<code>asmgen.py</code>	AST anotado	ASM con etiquetas
5. Resolución	<code>resolver.py</code>	ASM con etiquetas	ASM con offsets numéricos
6. Generación Binario	<code>binary_gen.py</code>	ASM resuelto	Archivo <code>.bin</code> + <code>.mem</code> (hex)

Invocación desde línea de comandos:

```
python main.py programa.yeison -b          # compilar a .bin
python main.py programa.yeison -b -S      # compilar + mostrar ASM
python main.py programa.yeison -b -m      # compilar + mapa de memoria
python main.py programa.yeison -v -l -s    # AST + tokens + tabla de
    simbolos
python main.py programa.yeison -a          # todo
```